AI 工具使用说明

项目：AIGC 互动大屏运营数据看板 **在线地址**：https://etan330.github.io/aigc-screen-ops-dashboard/

一、我使用了哪些 AI 工具

工具	我在哪些环节使用	具体用途
WorkBuddy（桌面端 AI 助手，Agent 模式，具备本地文件读写、命令执行与浏览器自动化能力）	全程	项目脚手架搭建、模拟数据生成脚本、指标计算、异常规则引擎、前端页面与组件、自动化测试、部署配置、文档初稿
Playwright（驱动本机 Google Chrome）	验收环节	以真实浏览器访问线上站点，检查首屏渲染、筛选联动、异常详情、设备详情与控制台报错
Vitest	验收环节	运行 25 项自动化测试，验证指标口径与异常识别
Python 3.13（标准库）	运行环境	执行模拟数据生成脚本

我没有在本项目中使用任何大模型 API、图像生成服务、数据库或后端框架；看板中的「AI 生成」是模拟数据里的事件，不是真实推理调用。

> 说明：本文档依据我实际提交的提示词、项目文件变更记录与验证输出整理。凡我无法从这些记录中确认的内容，我在第四节中标注为「待我确认」，没有代为补写。

二、关键提示词

整个项目我只提交了两轮主要提示词：第一轮完成产品原型，第二轮完成部署与交付。两轮提示词都是我自己写的完整需求说明，下面按原意摘录关键段落。

第一轮：产品原型开发

这一轮我把测评题目重新组织成一份可直接执行的需求，明确了业务场景、指标口径、异常规则、数据要求和交付边界。关键段落如下：

> **业务场景**：某品牌已上线多台 AIGC 互动大屏……完整链路为「用户参与 → 发起 AI 生成 → 生成成功或失败 → 内容安全审核 → 结果展示 → 用户扫码或分享」。运营人员需要同时管理活动效果、AI 服务质量、设备运行状态三个方面。
>
> **核心指标**：参与人数、AI 生成成功率、扫码转化率、分享转化率、平均生成时长、内容审核通过率、设备在线率。KPI 卡片除了数值，还应展示必要的业务上下文，例如分子与分母、与上一可比周期的变化或异常状态，避免只有一个孤立百分比。
>
> **口径要求**：优先从同一套原始模拟数据派生指标，不要为不同图表分别写死结果。生成成功率的分母只含成功与失败，进行中任务不计入；审核通过率的分母只含通过与拒绝，待审核不计入；扫码与分享转化率按内容去重，同一内容多次扫码或分享只计一次。默认把「生成成功且审核通过」的内容定义为可扫码、可分享的有效内容。请处理好分母为零、跨点位用户去重、跨日期任务、审核待处理等边界情况。
>
> **漏斗要求**：涉及漏斗时应统一统计对象和去重口径，不能将人数、任务次数、内容数混合计算为同一条转化漏斗。
>
> **异常规则**（演示阈值）：生成成功率低于 90% 且该点位当日已完成生成尝试不少于 20 次；平均生成时长超过 15 秒且当日成功任务不少于 20 次；内容审核拒绝率超过 10% 且已审核内容不少于 20 条；设备最近一次心跳距模拟数据基准时间超过 5 分钟判为离线。以上阈值属于演示规则，应清晰标注，方便后续修改。
>
> **异常展示要求**：每条异常应展示异常类型、严重程度、发生日期、涉及点位或设备、实际指标、阈值及样本量、可能的原因（明确属于待验证假设）、运营建议及下一步排查方向。不要只用静态红色标签假装实现异常监控，异常必须根据模拟数据和规则动态计算。
>
> **数据要求**：6–8 个运营点位、10–15 台设备、最近 30 天业务数据，优先采用事件级模拟数据。模拟数据必须有确定的随机种子，可以复现；同一指标在不同图表、详情页中计算结果应一致；异常需要在原始数据中有对应证据，不能只在 UI 上编造；数据不得使用真实用户隐私信息；明确统计截止时间。
>
> **交付要求**：React + Vite + TypeScript，Recharts 用于图表，lucide-react 用于图标，使用本地 JSON 模拟数据。README 面向没有开发经验的 macOS 用户；METRICS 一页以内；AI_USAGE 说明 AI 使用情况。完成开发后请实际执行安装、构建及必要的功能验证，修复可以发现的问题，不要将明显报错的项目直接作为交付物。
>
> **边界**：不允许以静态截图、设计稿、伪代码代替可运行产品；不允许为了视觉表现捏造与原始数据不一致的指标；不得虚构已经完成的测试、人工修改或项目部署。请先在内部完成业务模型、指标口径、数据结构及页面架构的设计，再进行编码。最后输出项目目录、运行方法、完成的功能、实际测试结果和未完成事项。

第二轮：部署与正式交付

第一版验收通过后，我提出了第二轮需求，核心是「不重新开发，只做部署与交付」：

> 第一版的整体设计、功能和视觉效果基本符合我的预期，因此本次不是重新开发产品，而是在现有项目基础上完成部署、验证与正式交付。
>
> **任务一**：优先使用 GitHub Pages，将现有项目部署为公开可访问的静态网站。点击链接后应直接进入完整的数据看板，而不是 GitHub 仓库、README、文件目录或代码预览页面。访问者不需要安装软件、运行命令、登录 GitHub。如果当前项目采用 React + Vite，优先使用现有工程构建静态网站，不需要为了追求单个 HTML 文件而重写产品。正确处理 GitHub Pages 的静态资源路径、项目子路径及页面刷新问题。部署完成后不应仅凭构建成功就认为任务完成，请实际访问公开网址验证，重点检查「本地运行正常，但 GitHub Pages 上页面空白、资源 404、数据加载失败」等部署问题。
>
> **任务二**：站在我作为招聘候选人的第一人称视角整理交付包。所有面向招聘方的材料都应从我作为候选人的视角撰写，而不是从 AI 助手、第三方评价者或系统执行者的视角撰写。README 是作品介绍，不是 AI 开发日志。
>
> **关于 AI 使用说明**：这是本次需要重点重写的文件。请统一改为第一人称。务必区分：我独立编写或直接修改的内容、我提出需求由 AI 执行的部分、AI 独立生成尚未经我确认的内容。不得将 AI 编写的代码描述为我亲自手写，也不得为了突出候选人能力虚构人工操作。如无法从现有记录中确认某项实际人工修改，请明确标记「待本人确认」。
>
> **边界**：不要推翻已有看板重新设计；不要为了单文件 HTML 强行放弃 React、图表库或现有交互能力；不要加入无关的新功能；不要虚构 GitHub 部署成功、访问验证成功或候选人的人工贡献；不要让最终提交包包含大量无关的缓存、构建中间文件或依赖目录。

我在这两轮之外做出的补充指令

除两轮主提示词外，我还在验收过程中提出了几条具体判断（均由 AI 执行）：

– 「模拟数据基准时间要固定，所有设备在线判断都相对这一基准时间计算，并且不能随历史日期筛选变化」——避免用历史区间误判当前在线状态； – 「异常里所有猜测性的原因必须标注为待验证假设，不能写成已确认根因」； – 「在线看板链接与源码仓库链接必须明确区分，不允许把 GitHub 仓库地址当作在线演示地址」； – 「提交包不要包含 node_modules、构建中间产物」； – 「README 里要直接给出在线体验入口，让招聘方打开仓库第一眼就能点进去」。

三、AI 辅助完成的主要内容

我把需求交给 AI 后，由 AI 执行的具体工作包括：

环节	AI 完成的工作
数据结构与生成	编写模拟数据生成脚本，定义「点位 → 设备 → 用户 → 参与事件 → 生成任务 → 内容 → 审核 → 扫码 / 分享」的 ID 关联链路，注入 5 类异常场景，输出 5 个模拟数据文件
指标计算	实现 7 项核心指标与日序列、点位、设备三个维度的聚合，处理去重、分母为零、待审核等边界
异常规则	实现 5 条异常的判定、严重度分级、样本量保护，以及「异常事实 → 对比 → 分布 → 原始样例 → 假设 → 行动」的证据下钻
页面与交互	搭建筛选栏、7 张 KPI 卡、四个业务区块、两个详情抽屉、演示案例区与口径折叠区，共 12 个组件与一套统一视觉样式
自动化测试	编写 25 项测试，覆盖指标口径与异常识别
部署配置	调整构建基础路径为相对路径、编写 GitHub Actions 工作流、创建并推送 GitHub 仓库、启用 GitHub Pages
文档	撰写 README、METRICS、AI_USAGE 的初稿（本文件已由我要求改为第一人称并重新组织）

需要说清楚的是：**本项目的代码由 AI 生成，不是我手写的。** 我负责的是提出需求、定义口径与验收标准、在关键节点做判断，以及对产出进行验收。第三节与第四节的区分就是我对这两类工作的划分。

四、我完成的主要修改与判断

4.1 由我提出并决定的内容

1）业务目标与交付形态

– 我确定了要交付的是**可直接打开使用的在线看板**，而不是代码仓库或本地项目，因此选择 GitHub Pages 静态部署； – 我判断不需要引入后端，因为测评场景下数据是模拟的、指标口径是固定的，纯前端已经足够，也能保证招聘方打开即可使用； – 我在第二轮明确要求「不重新开发」，保留第一版的产品形态，只做部署与交付整理——理由是第一版已经通过了我自己的验收，重做只会带来风险而不带来收益。

2）指标口径与边界

题目已经给出了核心口径要求。我的补充是把它们写清楚、写死，并要求实现时严格遵守：

– 生成成功率分母只含成功与失败，进行中任务不计入； – 审核通过率分母只含通过与拒绝，待审核不计入； – 扫码与分享按**内容**去重，同一内容多次行为只计一次； – 「生成成功且审核通过」才是有资格被扫码 / 分享的有效内容，审核拒绝的内容不进入转化率分母； – 明确时间归属：参与与生成按事件自身日期，内容 / 审核 / 扫码 / 分享统一归因到内容生成日，保证四步口径可比； – 分母为零时显示「—」而不是 0 或 NaN； – 所有指标必须从同一套原始数据派生，不允许为不同图表分别写死结果。

3）我设定的验收红线

– 不允许用静态截图或伪代码代替可运行产品； – 不允许为了视觉效果捏造与原始数据不一致的指标； – 异常必须在原始数据中有可核对的证据； – 不得虚构已完成的测试、人工修改或部署结果（这一条我也用在了本文件的写作要求上）。

4）部署与交付细节

– 选定仓库名与公开访问形式，要求在线地址打开即进入看板； – 要求区分「在线看板链接」与「源码仓库链接」，并说明各自用途； – 要求交付包不含 `node_modules`、`dist` 等无关文件； – 要求 README 直接给出在线体验入口，且写成作品介绍而不是开发日志。

5）材料视角的纠正

我发现第一版的 AI 使用说明带有「候选人提出…」「根据候选人的需求…」这类第三人称复盘口吻，不适合直接提交，因此明确要求整份文档改为第一人称，并要求区分「我做的判断」与「AI 执行的实现」。本文件就是这个要求的产物。

4.2 我提出要求、由 AI 执行完成的修改

以下内容我都参与了判断和验收，但**实现是 AI 完成的**，不是我亲手改的代码：

我提出的问题 / 要求	AI 的执行
第一版完成后，我要求实际执行安装、构建与功能验证并如实回报	AI 执行了安装、类型检查、25 项测试、生产构建与浏览器冒烟验证，并回报了结果
我要求设备在线状态不能随历史日期筛选变化	AI 将在线判定改为只相对模拟数据基准时间计算，并补充了对应测试
我要求异常必须能下钻到原始数据	AI 为每条规则实现了证据下钻（设备对比、原因分布、原始记录样例）
我要求样本文档中的每个数字都能在原始数据里找到对应	AI 用脚本直接统计原始数据核对，并据此修正了文档中的若干数值描述
我要求部署后必须实际访问线上地址验证，而不是只看构建成功	AI 用真实浏览器在无痕上下文（未登录 GitHub）访问线上站点，逐项检查渲染、筛选、详情与控制台报错

4.3 AI 独立生成、我尚未逐项复核的内容（待我确认）

以下内容是 AI 在设计过程中自行决定并实现的，我在验收时确认了它在页面上的表现是合理的，但**尚未逐项复核过实现细节**，如实标注如下：

– 异常严重度分档（严重 / 高 / 中）及其升级阈值（成功率 < 80%、时长 > 25s、拒绝率 > 20%、离线 > 60 分钟）；
– **R5「设备扫码转化率低于大盘」这条规则是 AI 补充的**，题目只要求了 4 类异常；我认可它有价值并保留在演示中，并已在文档里标注为「扩展演示规则」； – 风格维度拒绝率采用的「相对当日大盘的加权平均偏离」这一统计口径； – 转化漏斗拆成「用户层 / 内容层」两层的具体呈现方式； – 两个演示案例（案例 A / 案例 B）的具体措辞与行动建议； – 数据生成器中各点位的基线客流、风格偏好权重等具体参数。

4.4 需要说明的一点

本项目没有「我手工编辑代码」的记录。所有代码、配置与部署工作流都由 AI 生成与执行；我做的是需求定义、口径判断、验收与材料组织。我不想把 AI 的产出描述成我的手工劳动。

如果后续我在本地对代码或文档做了手工调整，我会在此基础上补充记录。

五、验证与反思

5.1 实际执行过的验证

以下验证均在本次交付过程中实际执行过，命令与输出可复现：

验证项	方式	结果
类型检查	`tsc --noEmit`（strict 模式）	通过，0 error
自动化测试	`npm test`（Vitest）	25 项全部通过：指标口径 16 项 + 异常规则 9 项
生产构建	`npm run build`	成功，产物 674 kB JS（gzip 194 kB）+ 14.8 kB CSS
子路径兼容	本地把构建产物挂在 `/aigc-screen-ops-dashboard/` 子路径下访问	页面与数据正常加载，无 404
线上可访问	浏览器访问公开地址	HTTP 200，看板完整渲染
线上功能回归（无痕上下文、未登录 GitHub）	真实浏览器逐项操作	7 张 KPI 卡数值与本地一致；12 行设备；33 个图表正常渲染；日期与点位筛选联动；异常详情与设备详情抽屉可打开；控制台与网络 **0 报错**
数据一致性	测试中比对日序列求和、点位拆分求和、设备拆分求和与总汇总	完全一致

测试覆盖的关键口径包括：生成中任务不进分母、待审核不进分母、同一内容多次扫码只计一次、分享失败不计入、跨点位用户只计一次、分母为零返回 `–`、设备在线判定不随日期变化，以及 5 条异常规则的触发与不触发。

5.2 尚未验证的事项

– 只在 Google Chrome 上做过完整验证，未覆盖 Safari、Firefox；未做移动端适配（布局最小适配到 900px）。 – 未做性能压测。事件数据约 10MB、纯前端聚合，数据量再大一个量级需要改成服务端聚合。 – 未验证真实扫码链路的落地页加载情况——这部分没有模拟数据支撑，案例 B 中相关判断已标注为「需服务端埋点验证」。

5.3 关于模拟数据与静态部署的取舍

这个项目是测评场景，没有真实业务数据，也不应该使用任何真实用户信息。因此我用固定随机种子的脚本生成了一整套事件级模拟数据，好处是：

– 结果可复现，招聘方重新生成数据后看到的指标与我提交的完全一致； – 指标之间的勾稽关系真实存在（同一个用户、同一条内容能在多个页面被关联到），可以验证口径是否自洽； – 我能在数据里预先埋入几类异常，让「异常监控」这个最重要的功能可以被真实演示和核对，而不是靠截图说明。

静态部署则是为了让招聘方零成本查看作品：不需要安装环境、不需要运行命令、不需要登录。代价是没有服务端，数据只能随前端一起发布。

5.4 如果放到真实生产环境，还需要补什么

– **数据链路**：把前端聚合换成服务端预聚合接口与指标层，接入真实埋点（参与、生成、审核、扫码、分享、心跳），并统一事件 ID 规范； – **实时设备状态**：心跳走独立通道并做去抖动，避免网络抖动被误判为离线； – **告警与闭环**：异常规则接入告警通知与工单流转（派单给技术运维或审核策略），让「发现异常 → 采取行动」真正闭环，而不是停留在看板上； – **权限与合规**：运营、审核、运维的角色与数据权限分离；内容审核相关数据需要脱敏与留痕。

这些是当前版本明确没有覆盖的部分，也是我理解这个产品往下走最需要补齐的能力。